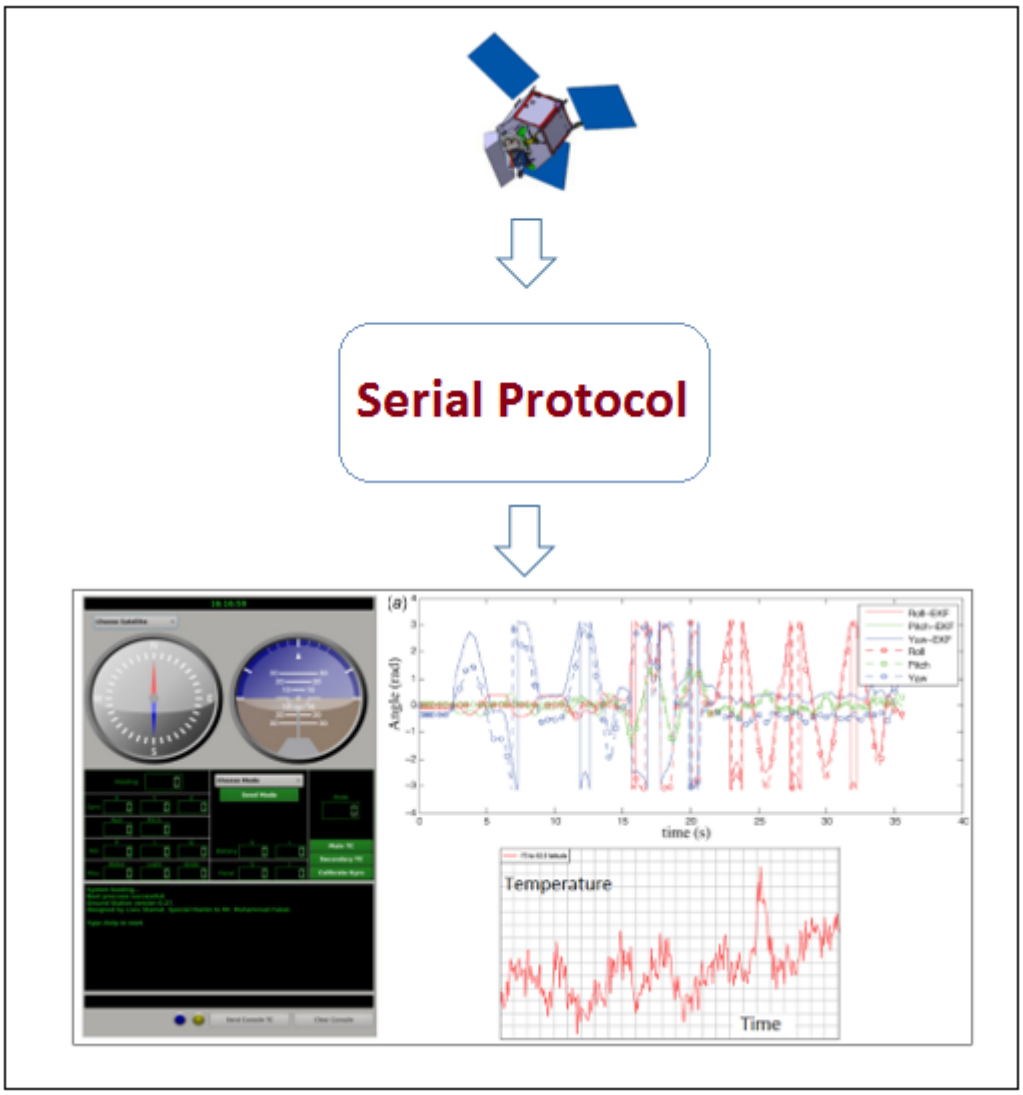
Julius-Maximilians-
UNIVERSITÄT
WÜRZBURG

Chair of Computer Science VIII
Aerospace Information Technology
Prof.Dr.Sergio Montenegro.

Luft & Rumfahrt Labor: Development of a Satellite's Ground Station



Contents

1 Introduction **3**

2 Prerequisite **3**

3 Download Qt **3**

4 SatelliteSide (Microcontroller) Programming **3**
4.1 Task1 4
4.2 Telemetry 4
4.2.1 Task2:Attitude Heading Reference Systems and Spacecraft Temperature 4

5 GroundSattionSide Programming **5**
5.1 Library to Use SerialPort in Qt 5
5.2 Signals and Slot for Serial Port API 5
5.3 Data Reception 6
5.4 Data Transmission 7
5.5 Important Functions for Qt Programming 7
5.6 Task3: Telecommand 7
5.6.1 Task3a:Command for Calibration 7
5.6.2 Task3b: Command for starting the Telemetry 8
5.7 Adding the widget (graphical Tool) on the User Interface (.ui) file in Qt 8

6 Adding the Graph (line Graph) to display the temperature with respect to time on the User Interface (.ui) file in Qt **13**

1 Introduction

A ground station, earth station, or earth terminal is a terrestrial radio station designed for extraplanetary telecommunication with spacecraft (constituting part of the ground segment of the spacecraft system), or reception of radio waves from astronomical radio sources.[1]

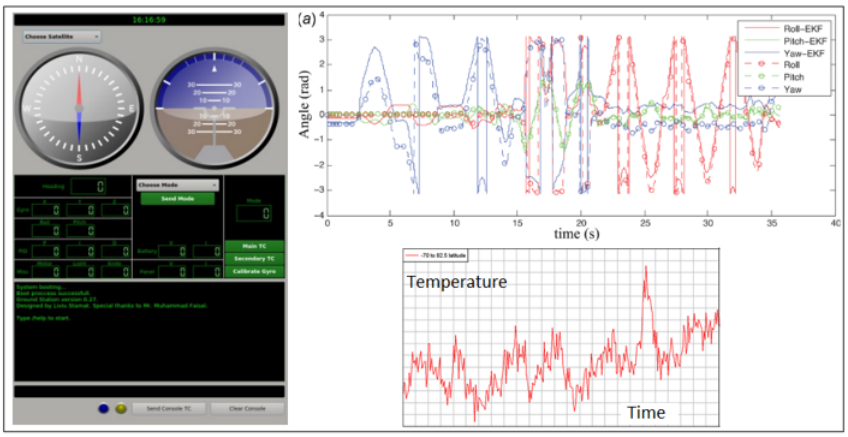


Figure 1

2 Prerequisite

- Eclipse Toolchain for Stm32f407.
- Qtcreator (newest version would be recommended)
- Working Embedded code to get the IMU data.
- Handling of Telecommands on the side of on–board computer.

3 Download Qt

- Qt with Qt creator can be downloaded from the following site, make sure that you down the open–source version.
- `https://www.qt.io/download`

4 SatelliteSide (Microcontroller) Programming

In this exercise there will be a telemetry thread and a telecommand thread running on the satellite controller, The telemetry thread publishes (transmits) the IMU data (x,y,z components of Gyro, Accelerometer, Magnetometer, Pitch , Roll, Yaw and the Temperature), The telecommand thread monitors all the incoming commands from the groundstation to start or stop the data.

4.1 Task1

1. Set the the parameters of Telemetry and Telecommand in their threads and also define the IDs for topics.
2. The strucuture of the telecommand frame should have the same parameters as the message body of the Telecommand in exercise 3 i.e
   "$ < X >< data > #" whereas:
   - " $ " is the Message Start Character [1 byte].
   - X is a predefined Message ID character [1 byte].
   - data are ASCII characters which correspond to a valid floating point number [0 - 10 bytes].
   - "#" is the Message End Character [1*byte*].
3. Write the telemetry and telecommand thread as described above in the section 4

4.2 Telemetry

4.2.1 Task2:Attitude Heading Reference Systems and Spacecraft Temperature

- Transmit the x,y and z components of Gyro, Accelerometer and Magnetometer.
- Pitch, Roll any yaw of the satellite.
- On–Board temperature.

```
TIME_LOOP(NOW(),1*SECONDS)

{
    PRINTF("%c%c%lf%c",start,topic1,temperature,end);

    suspendCallerUntil(NOW()+0.5*SECONDS);

    PRINTF("%c%c%lf%c%lf%c%lf%c",start,topic1,f1,',',f2,',',f3,end);

}
```

Figure 2: Telemetry Thread

In the above figure, The tranmist method is described, The float values are separated by ',' . It helps to extract the floating point values on the groundstation side. These float values represents the x,y,z components of either Gyro, Accelerometer or Magnetometer.

5 GroundSattionSide Programming

A raw template has been provided to make the Graphical User Interface of the ground station, It is based on a C++ library Qt.

5.1 Library to Use SerialPort in Qt

In this template a Qt Library QSerialport is used to handle the serial communicaiton. In the following figure the objects of the related QSerialport classes are defined and also the declaration of the methods of reading and writing the data are also shown here.

```
private:
    Ui::MainWindow *ui;
    QString portName;
    QString baudrate;
    QSerialPort   *serial;

public slots:
void readData();
void writeData1();
void connectUart(void);
```

Figure 3: Qserial port API

5.2 Signals and Slot for Serial Port API

- connectuart fuction (slot) is connected with a button present on the GUI to initialize the serial port with proper COM port and baudrate.
- readData slot is connected with the readyRead signal defined in QSerialport class. This signal will be automatically fired whenever some data is present at the serialport.
- writeData1 function constructs the telecommand message and transmit it whenever the send button is pressed.

```
connect(ui->connectbtn,SIGNAL(clicked()),this,SLOT(connectUart()));

connect(serial, &QSerialPort::readyRead, this, &MainWindow::readData);

connect(ui->send,SIGNAL(clicked()),this,SLOT(writeData1()));
```

Figure 4: Signals and slots for the serialport communication

5.3 Data Reception

In the readData fuction the methods to extract the data should be defined which should be based on the ID of the message as shown below.

```
void MainWindow::readData(void)
{
  char start,topic,end,separator1,separator2;
  double f1,f2,f3;
  QByteArray bdata;
  bdata = serial->readAll();

  QString serialBuffer = QString :: fromStdString(bdata.toStdString());
  QTextStream str(&serialBuffer);
  qDebug()<<serialBuffer<<endl;

  str >> start >> topic >>f1 >> separator1 >> f2 >> separator2 >> f3>> end;

  qDebug()<<"start"<<start<<endl;
  qDebug()<<"topic"<<topic<<endl;
  qDebug()<<"f1="<<f1<<endl;
  qDebug()<<"f2="<<f2<<endl;
  qDebug()<<"f3="<<f3<<endl;
  qDebug()<<"end="<<end<<endl;


  serial->flush();
  serial->clear();

}
```

Figure 5

- The incoming data is stored in an array of bytes as shown in blue box.
- This byte's array is then converted into normal string.
- This string is used to make textstream to extract the different quantities present in the telemetry frame as shown in the red box.

5.4 Data Transmission

Before transmitting the telecomannds the whole message should be packed in a string with the same structure as the telemetry, It is described below. This telecommand has to be recieved and processed by the on–board computer of the satellite.

```
 void MainWindow::writeData1()
{

    char baData[50];

    sprintf(baData,"%c%c%f%c",'$','T',1.00,'#');
    serial->write(baData);
}
```

Figure 6

5.5 Important Functions for Qt Programming

(a) How many bytes are read:
- QByteArray::size()
- QByteArray::length()

(b) How to find the Start and the last character to Check the validity of the incoming message .
- QByteArray::at(0)
- QByteArray::at("No of last Byte");

5.6 Task3: Telecommand

5.6.1 Task3a:Command for Calibration

There should be one telecommand to perform the calibration of IMU on the satellite.

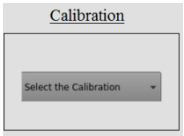


Figure 7: Calibration Control

5.6.2 Task3b: Command for starting the Telemetry

There should be an option to switch–On and switch–Off the telemetry as shown below:

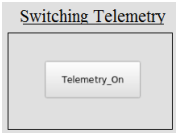


Figure 8: Telemetry Command

- You have to use Qt's signal and slot mechanism to display the data on the GUI and to send the telecommand on the serial port.
- Your GUI should look similar to the interface as given below:

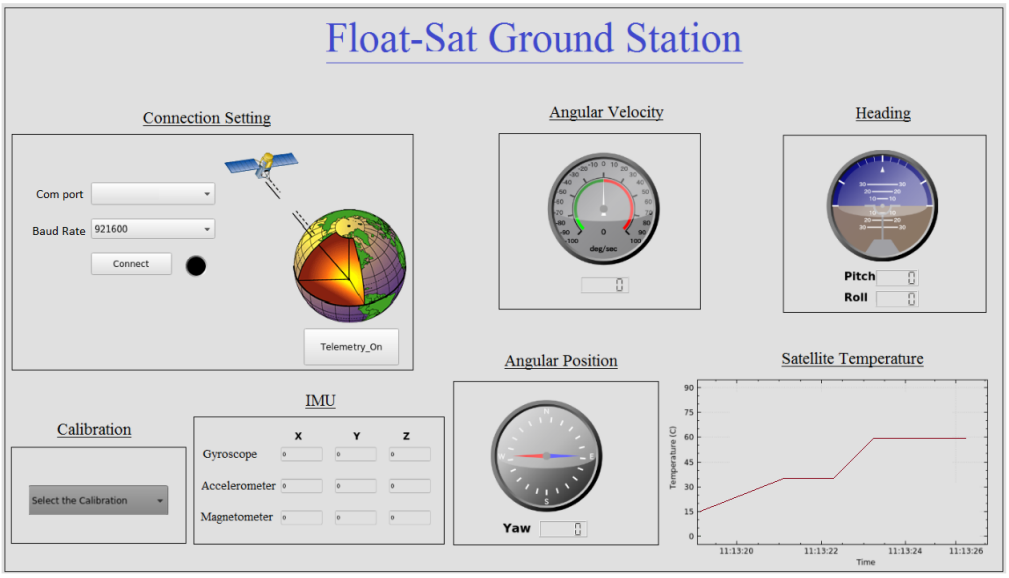


Figure 9

5.7 Adding the widget (graphical Tool) on the User Interface (.ui) file in Qt

Sometimes a uer uses the third party widgets to add on its GUI, the third party mostly offers the widgets in the form of the code so the user has to visualize the tool from the third party code into his GUI window. In the following a visual method is described to use a similar third party API QFlightinstruments to display the widgets related to the IMU data. This API is provided to you with the template of the exercise.

- On the form file (mainwindow.ui) from tool box select tool "widget" and put it on your form window as shown below

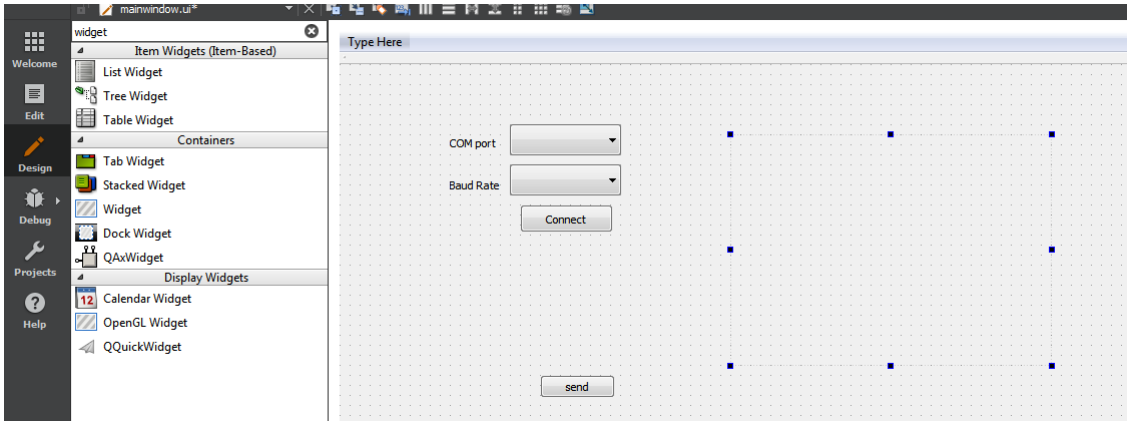


Figure 10

- Right click on this widget and click "promote to" and type the class name "QADI" and the name of the header file where the class is declared (qFlightInstruments.h) then click add and then click promote as shown below:

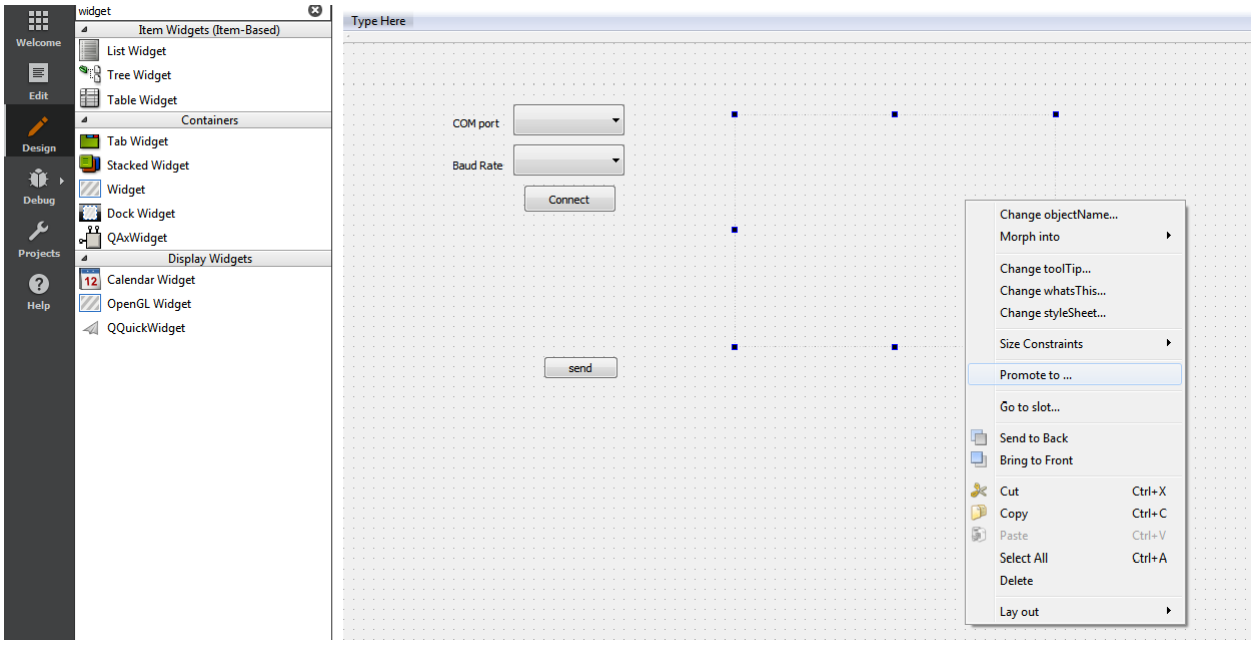


Figure 11

- Name your added widget as "attitudeIndicator"' in the object box as shown below:

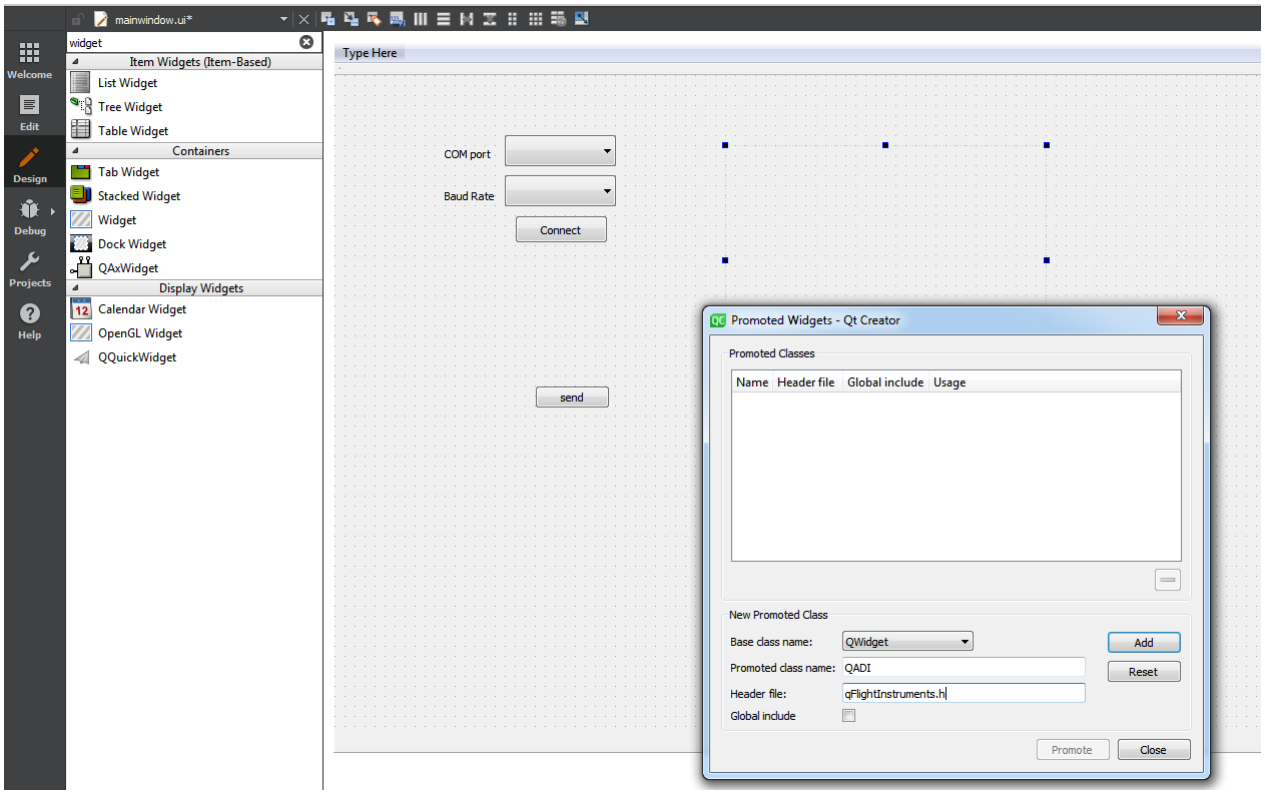

mainwindow.ui*
widget
Welcome
Edit
Design
Debug
Projects
Help
Item Widgets (Item-Based)
List Widget
Tree Widget
Table Widget
Containers
Tab Widget
Stacked Widget
Widget
Dock Widget
QAxWidget
Display Widgets
Calendar Widget
OpenGL Widget
QQuickWidget
Type Here
COM port
Baud Rate
Connect
send
Promoted Widgets - Qt Creator
Promoted Classes
Name
Header file
Global include
Usage
New Promoted Class
Base class name:
QWidget
Promoted class name:
QADI
Header file:
qFlightInstruments.h
Global include
Add
Reset
Promote
Close

Figure 12

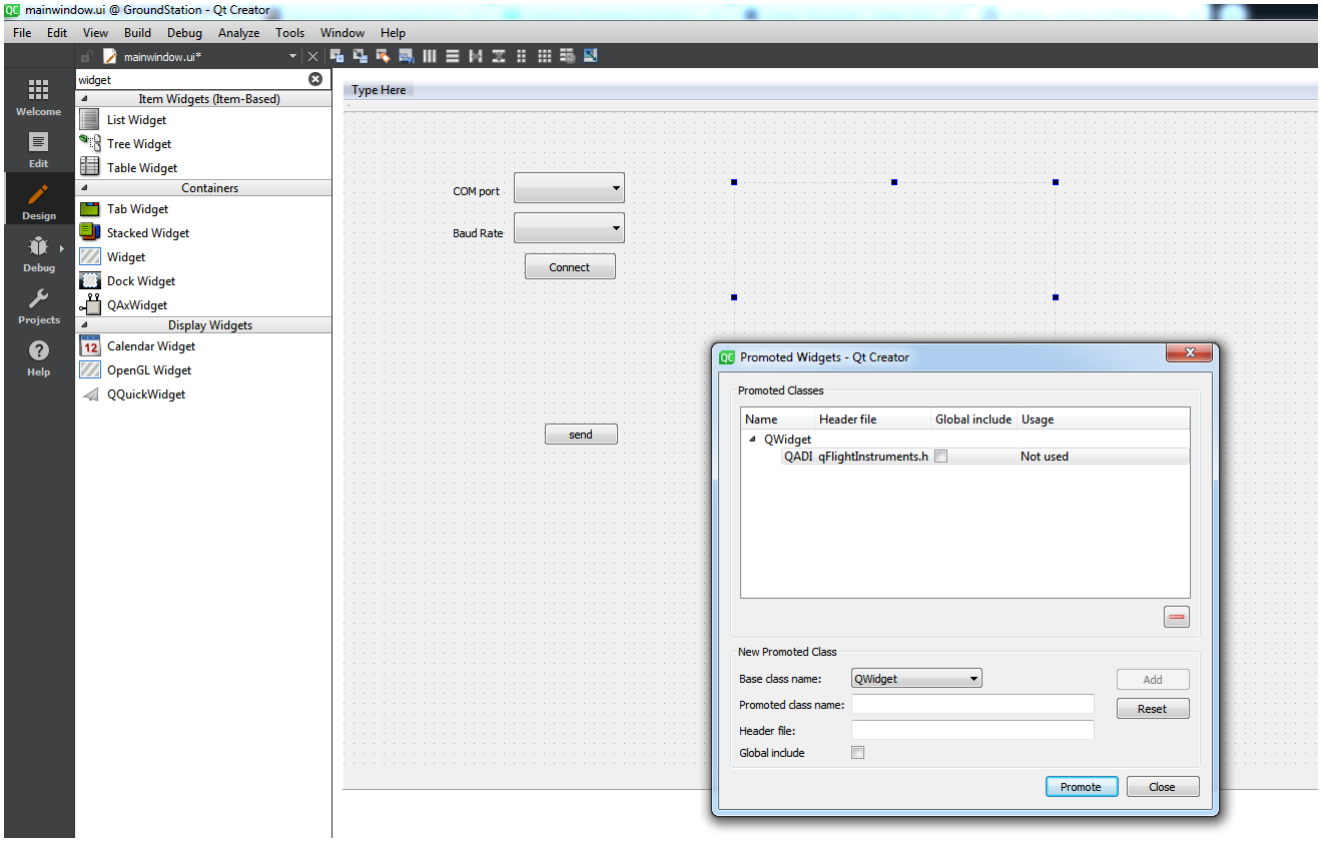


Figure 13

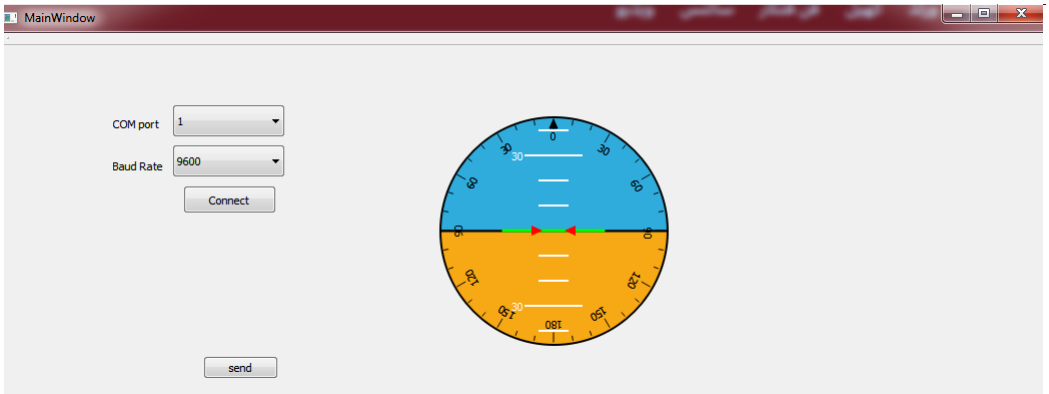


Figure 14

- Name this widget as “attitudeIndicator”

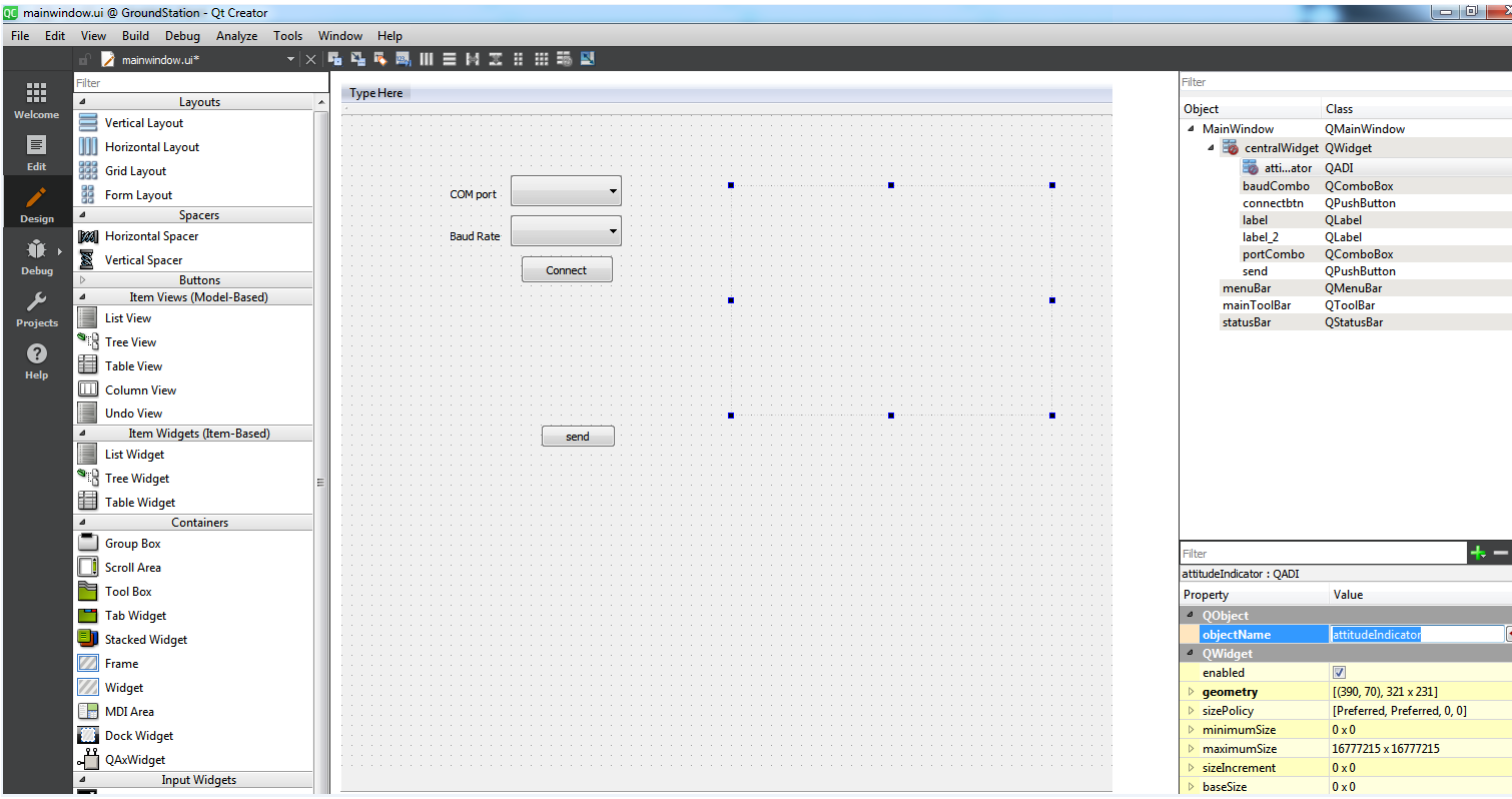


Figure 15

- Add one more widgets as we did in last 3 steps and name the second widget widget as “compassIndicator”
- Promote this widget to QCompass class as declared in qFlightInstruments.h file.
- Now build the project and run the application you should get the output as shown below :

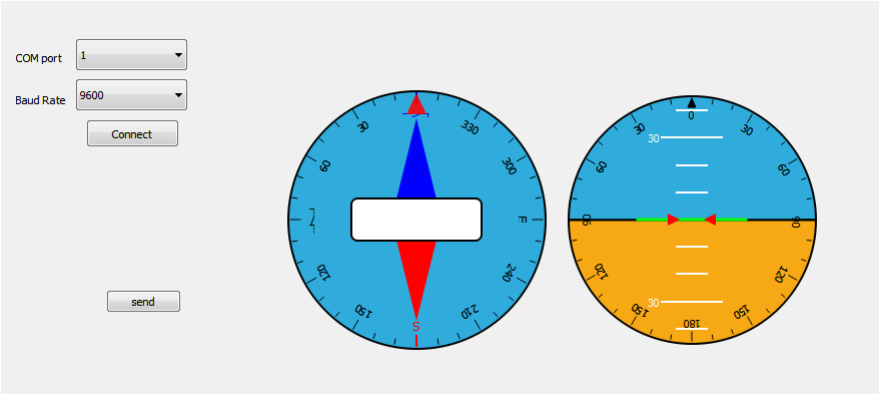


Figure 16

6 Adding the Graph (line Graph) to display the temperature with respect to time on the User Interface (.ui) file in Qt

In order to display a runtime plot for the temperature another opensource library QCustomPlot is used.

- Place a regular QWidget on your form in the desired location. Right click on it and hit Promote to...

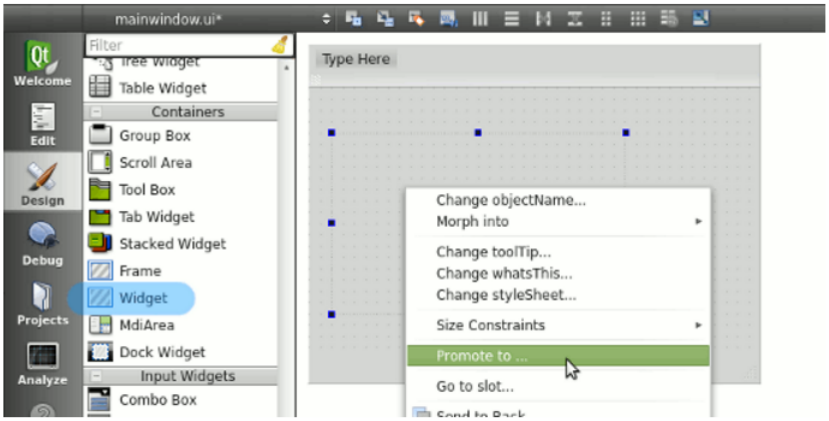


Figure 17

- In the following dialog box enter QCustomPlot in the input field next to Promoted class name. The input next to Header file should automatically fill with the correct qcustomplot.h value. Hit Add to add QCustomPlot to the promoted classes list and finally hit Promote to turn the QWidget on your form into a QCustomPlot.

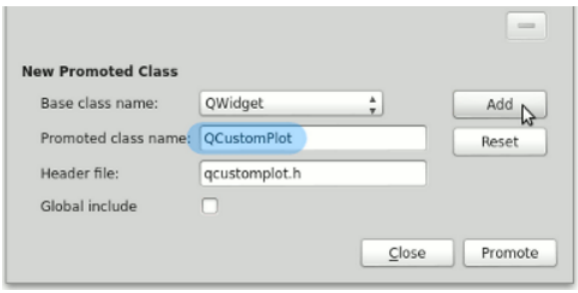


Figure 18

- You won't see any immediate visual changes in QtCreator (or QtDesigner), but while running the application, you will see an empty plot with axes and grid lines as shown below:

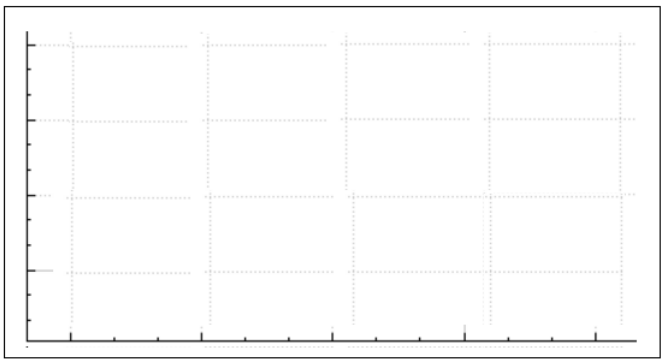


Figure 19

- You have been provided a graphsource.cpp file where you can get all the required code to plot the temperature versus time.

The information about the library of analog widget (Attitude widgets) can be obtained from [?] and the source website for the qtcustomplot (graph) library is [2]

References

[1] Ground station in Wikipedia

[2] QCustomPlot is a Qt C++ widget for plotting and data visualization.
http://qcustomplot.com/

[3] https://www.mate-look.org/browse/cat/252/page/2/ord/latest/